

EXPERIMENT-4

EXPERIMENT-4(a)

Student Details

Name: R.Varshitha

Roll Number: 24501A4450

Experiment Title: Simple Linear Regression using Air Quality Dataset

Dataset: Air Quality Dataset

Algorithm: Simple Linear Regression

1. Business Understanding

1.1 Problem Statement & Relevance

Air pollution has become one of the major environmental issues affecting human health. Predicting air quality parameters helps government agencies and environmental organizations monitor pollution levels and take preventive measures.

In this experiment, a **Simple Linear Regression** model is developed to predict **Carbon Monoxide (CO(GT)) concentration** using **Temperature (T)** as the independent variable.

Problem Statement

Develop a machine learning model that predicts CO(GT) concentration based on temperature measurements.

Relevance

- Helps monitor pollution levels.
- Supports environmental decision making.
- Assists in predicting air quality.
- Useful for smart city applications.

1.2 Expected Benefits

- Early prediction of pollution.
- Better environmental monitoring.
- Reduced health risks.
- Improved policy planning.
- Supports research in environmental science.

2. Data Understanding

2.1 Dataset Structure

Dataset Name: **Air Quality Dataset**

Number of Records: Approximately **9358**

Number of Attributes: **15**

Target Variable:

CO(GT)

Independent Variable:

Temperature (T)

2.2 Data Dictionary

Column	Description	Type
Date	Observation Date	Object
Time	Observation Time	Object
CO(GT)	Carbon Monoxide Concentration	Float
PT08.S1(CO)	CO Sensor Response	Integer
NMHC(GT)	Non-Methane Hydrocarbons	Float
C6H6(GT)	Benzene Concentration	Float
PT08.S2(NMHC)	NMHC Sensor Response	Integer
NOx(GT)	Nitrogen Oxides	Float

PT08.S3(NOx)	NOx Sensor Response	Integer
NO2(GT)	Nitrogen Dioxide	Float
PT08.S4(NO2)	NO2 Sensor Response	Integer
PT08.S5(O3)	Ozone Sensor Response	Integer

T	Temperature	Float
RH	Relative Humidity	Float
AH	Absolute Humidity	Float

2.3 Data Quality Observations

#import libraries Source

Code:

```
import pandas as pd
import matplotlib.pyplot as plt
import numpy as np
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
print("Libraries Imported")
```

Output: Libraries Imported

#Load Dataset Source Code: df =

```
pd.read_csv('/content/Air Quality.csv')
print("Dataset reading complete")
```

Output: Dataset reading complete

#Display 5 rows

df.head() Output:

	Date	Time	CO(GT)	PT08.S1(CO)	PM10(GT)	PM10.S2(NMHC)	NOx(GT)	PT08.S3(NOx)	NO2(GT)	PT08.S4(NO2)	PT08.S5(O3)	T	RH	AH	Unnamed: 15	Unnamed: 16	
0	10/03/2004	18:00:00	2.6	1360.0	150.0	11.9	1046.0	166.0	1056.0	113.0	1692.0	1268.0	13.6	48.9	0.7578	NaN	NaN
1	10/03/2004	19:00:00	2.0	1292.0	112.0	9.4	955.0	103.0	1174.0	92.0	1559.0	972.0	13.3	47.7	0.7255	NaN	NaN
2	10/03/2004	20:00:00	2.2	1402.0	88.0	9.0	939.0	131.0	1140.0	114.0	1555.0	1074.0	11.9	54.0	0.7502	NaN	NaN
3	10/03/2004	21:00:00	2.2	1376.0	80.0	9.2	948.0	172.0	1092.0	122.0	1584.0	1203.0	11.0	60.0	0.7867	NaN	NaN
4	10/03/2004	22:00:00	1.6	1272.0	51.0	6.5	836.0	131.0	1205.0	116.0	1490.0	1110.0	11.2	59.6	0.7888	NaN	NaN

#Display the last 5 rows of dataset

df.tail() Output:

	Date	Time	CO(GT)	PT08.S1(CO)	PM10(GT)	PM10(S2(NMHC))	NOx(GT)	PT08.S3(NOx)	NO2(GT)	PT08.S4(NO2)	PT08.S5(O3)	T	RH	AH	Unnamed: 15	Unnamed: 16
9466	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN
9467	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN
9468	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN
9469	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN
9470	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN

Source Code:

df.shape Output:

(9471, 17)

Summary of a dataframe

Df.info

Output:

```
int_col text_col float_col
0      1    alpha    0.00
1      2    beta    0.25
2      3    gamma    0.50
3      4    delta    0.75
4      5  epsilon    1.00
```

Missing Values

Some columns contain missing values represented as **-200**.

Duplicate Values

Duplicates should be removed if found.

#Detect missing values Source

Code:

```
df.isnull().sum()
```

Output:

```
0
CO(GT)      0
PT08.S1(CO)  0
NMHC(GT)    0
C6H6(GT)    0
PT08.S2(NMHC) 0
NOx(GT)     0
PT08.S3(NOx) 0
NO2(GT)     0
PT08.S4(NO2) 0
PT08.S5(O3)  0
T           0
RH          0
AH          0
```

dtype: int64

Outliers

Outliers exist because pollution values vary greatly.

Data Type Conversion

Date and Time columns remain object type.

Numerical columns are converted into float values.

3. Data Preparation

3.1 Data Cleaning

Performed the following preprocessing steps:

- Loaded dataset.
- Removed unwanted columns.
- Replaced -200 with NaN.
- Removed missing values.
- Selected required columns.
- Converted numerical columns into float.

3.2 Feature Engineering

Independent Variable (X)

Temperature (T)

Source Code:

```
# Define the independent variables (features)
X = df.drop(columns=['CO(GT)'])
X
```

Output:

	PT08.S1(CO)	NMHC(GT)	C6H6(GT)	PT08.S2(NMHC)	NOx(GT)	PT08.S3(NOx)	NO2(GT)	PT08.S4(NO2)	PT08.S5(O3)	T	RH	AH
0	1360.0	150.0	11.9	1046.0	166.0	1056.0	113.0	1692.0	1268.0	13.6	48.9	0.7578
1	1292.0	112.0	9.4	955.0	103.0	1174.0	92.0	1559.0	972.0	13.3	47.7	0.7255
2	1402.0	88.0	9.0	939.0	131.0	1140.0	114.0	1555.0	1074.0	11.9	54.0	0.7502
3	1376.0	80.0	9.2	948.0	172.0	1092.0	122.0	1584.0	1203.0	11.0	60.0	0.7867
4	1272.0	51.0	6.5	836.0	131.0	1205.0	116.0	1490.0	1110.0	11.2	59.6	0.7888
...	...	...	...	...	...	...	...	...	...	...	...	...
1226	1449.0	501.0	19.5	1282.0	254.0	625.0	133.0	2100.0	1569.0	19.1	61.1	1.3345
1227	1363.0	234.0	15.1	1152.0	189.0	684.0	110.0	1951.0	1495.0	18.2	65.4	1.3529
1228	1371.0	212.0	14.6	1136.0	174.0	689.0	102.0	1927.0	1471.0	18.1	66.1	1.3579
1229	1406.0	275.0	13.7	1107.0	167.0	718.0	108.0	1872.0	1384.0	17.7	66.9	1.3422
1230	1425.0	275.0	15.2	1155.0	185.0	709.0	110.0	1936.0	1789.0	17.8	66.8	1.3460

827 rows × 12 columns

Dependent Variable (Y)

Carbon Monoxide (CO(GT))

Source Code:

```
# Define the dependent variable (target) y
= df['CO(GT)']
Y
```

Output:

```
      CO(GT)
0         2.6
1         2.0
2         2.2
3         2.2
4         1.6
...
1226        4.4
1227        3.1
1228        3.0
1229        3.1
1230        3.5
127 rows × 1 columns

dtype: float64
```

3.3 Train-Test Split

Training Data = 80%

Source Code:

```
#Split the dataset into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
random_state=42) print("Dataset cleaned and split into training and
testing sets.") print(f"X_train shape: {X_train.shape}") print(f"X_test
shape: {X_test.shape}") print(f"y_train shape: {y_train.shape}")
print(f"y_test shape: {y_test.shape}") Output:
```

```
Dataset cleaned and split into training and testing sets.
X_train shape: (661, 12)
X_test shape: (166, 12)
y_train shape: (661,)
y_test shape: (166,)
```

Testing Data = 20% **Source**

Code:

```
train the model y_pred =
model.predict(X_test)
```

Random State = 42

4. Modelling

4.1 Model Selection

Algorithm Used: **Simple**

Linear Regression

Reason:

Simple Linear Regression predicts a dependent variable using one independent variable by fitting the best straight line.

Regression Equation:

$$\text{CO(GT)} = \beta_0 + \beta_1 \times \text{Temperature}$$

4.2 Model Training

Steps:

1. Import libraries.
2. Load dataset.
3. Clean dataset.
4. Select Temperature as X.
5. Select CO(GT) as Y.
6. Split data.
7. Train Linear Regression model.
8. Predict testing values.

#generate predictions for training and testing datasets using the trained model **Source Code:**

```
y_train_pred = model.predict(X_train) y_test_pred
= model.predict(X_test)
print("Predictions generated for training and testing datasets.")
print(f"y_train_pred shape: {y_train_pred.shape}")
print(f"y_test_pred shape: {y_test_pred.shape}") Output:
    Predictions generated for training and testing datasets.
    y_train_pred shape: (661,)
    y_test_pred shape: (166,)
```

4.3 Model Evaluation & Interpretation

Performance Metrics:

- Mean Absolute Error (MAE)
- Mean Squared Error (MSE)
- Root Mean Square Error (RMSE)
- R^2 Score

Interpretation:

- Lower MAE indicates better prediction.
- Lower RMSE indicates lower prediction error.
- Higher R^2 indicates better model fit.

```
#evaluate the model using metrics  
#evaluate the model using metrics
```

- **Source Code:**

```
mae = mean_absolute_error(y_test, y_test_pred)  
mse = mean_squared_error(y_test, y_test_pred)  
rmse = np.sqrt(mse) r2 = r2_score(y_test,  
y_test_pred) print(f"Mean Absolute Error (MAE):  
{mae}") print(f"Mean Squared Error (MSE):  
{mse}") print(f"Root Mean Squared Error (RMSE):  
{rmse}") print(f"R-squared (R2) Score: {r2}")
```

Output:

Output:

```
Mean Absolute Error (MAE): 0.17794055006131818  
Mean Squared Error (MSE): 0.0579252942473791  
Root Mean Squared Error (RMSE): 0.24067674222362886  
R-squared (R2) Score: 0.9721733275987302
```

5. Evaluation

5.1 Appropriate Metrics Selection

The following metrics are used:

MAE

Average prediction error.

MSE

Average squared error.

RMSE

Square root of MSE.

R² Score

Measures model accuracy.

Ideal value is close to 1.

#evaluate the model using metrics for training the data **Source**

Code:

```
mae_train = mean_absolute_error(y_train, y_train_pred)
mse_train = mean_squared_error(y_train, y_train_pred)
rmse_train = np.sqrt(mse_train) r2_train =
r2_score(y_train, y_train_pred)
print(f"Mean Absolute Error (MAE) for Training Data: {mae_train}")
print(f"Mean Squared Error (MSE) for Training Data: {mse_train}")
print(f"Root Mean Squared Error (RMSE) for Training Data: {rmse_train}")
print(f"R-squared (R2) Score for Training Data: {r2_train}") Output:
```

```
Mean Absolute Error (MAE) for Training Data: 0.1621082553114735
Mean Squared Error (MSE) for Training Data: 0.04810562864093506
Root Mean Squared Error (RMSE) for Training Data: 0.2193299538160145
R-squared (R2) Score for Training Data: 0.9754261167212304
```

5.2 Interpretation

The regression model predicts Carbon Monoxide concentration using Temperature.

If the R² score is high and RMSE is low, the model performs well.

6. Conclusion

The Air Quality dataset was successfully analyzed using **Simple Linear Regression**.

The model learned the relationship between **Temperature** and **Carbon Monoxide concentration**.

The developed model can assist environmental monitoring systems in estimating pollution levels.

Future improvements include:

- Multiple Linear Regression
- Decision Tree Regression
- Random Forest Regression
- XGBoost Regression

7. Appendix

#Plot actual training data points vs. predicted values for visualization

Source Code: plt.figure(figsize=(10, 6))

```
sns.scatterplot(x=y_train, y=y_train_pred, alpha=0.6)
plt.plot([y_train.min(), y_train.max()], [y_train.min(), y_train.max()], 'r--', lw=2) # Line for perfect prediction
plt.title('Actual vs. Predicted CO(GT) (Training Set)')
plt.xlabel('Actual CO(GT)') plt.ylabel('Predicted CO(GT)') plt.grid(True) plt.show()
```

Output:


